



CROWDFUNDING

FALL 2025 CAPSTONE PROJECT | PROF HENRY WONG

BY

TEAM NEXUS

AGENDA

1. Team Member Roles | Improvements | Responsibilities
2. Problem Statement | Project Description | Team Working Agreement
3. User Personas | MVP | Project Design
4. Technologies | Algorithms | Diagrams
5. Product Backlog
6. Sprint Summary
7. Metrics
8. Sprint 2 Recap
9. Sprint 3
 - User Stories
 - Test Cases
 - Stories Completed
 - Stories Not Completed
9. Metrics
10. Retrospective
11. Project Demo

GETTING TO KNOW THE TEAM...



Ashwanth Maram
Full Stack Developer



Ashwitha Maram
Developer/Tester



Raj Mandaliya
Block Chain Developer

GETTING TO KNOW THE TEAM...



Vinukonda Yashwanth Reddy
Backend and Database Engineer



Nyi Nyi Lwin
Full Stack Developer



Naveen Pesalabanda
SCRUM Master

IMPROVEMENTS

- Improved Algorithms section
- Improved sprint backlog and product backlog
- Improved test cases
- Improved the Metrics (Team Velocity, Sprint Burn down Chart, Completed Committed Ratio)
- Improved project demo section



PROJECT DESCRIPTION

Project Name:	Crowd Funding
Team:	Team Nexus
Project Description:	It's a platform that helps people raise money safely and fairly using blockchain, making sure all funds are transparent and controlled by the community. For campaign creators and contributors who seek a transparent, secure, and fair way to raise and fund projects, the Crowd Funding platform is a blockchain-based fundraising solution that enables campaign creation, contributor governance, and automatic refunds. Unlike traditional centralized platforms like Kickstarter or GoFundMe, our application uses smart contracts to eliminate intermediaries, reduce fees, and ensure community-driven decision-making.
Benefit Outcomes:	Transparent funds – everyone can see usage Secure money handling – via blockchain Community voting – contributors decide Automatic refunds – if project fails Low fees – cheaper than traditional platforms
GitHub Link:	https://github.com/htmw/F2025-Team2/wiki

TEAM WORKING AGREEMENT

Communication Expectations

- Use Zoom, GitHub for collaboration.
- Respect everyone opinions; decisions are made collectively.
- Notifying the team in advance if you cannot complete or attend.
- In case of any conflicts, present research-backed solutions and resolve by majority vote.
- Keeping everyone updated when pushing code or making changes.
- No last-minute dumping of work on others.

Meeting Schedules

- Meetings will be held twice weekly on Zoom or Google meet
- Meeting time is set based on availability.
- Agenda will be shared in advance for preparation.
- Sprint Planning at the start, Retrospective at the end of each sprint.
- Absence must be informed ahead of time; member must read meeting summary.
- All members are expected to participate and contribute fairly.

TEAM WORKING AGREEMENT

Team Dynamics & Principles

- Each member must clearly understand and own their tasks.
- Workload distributed fairly; large tasks broken into smaller ones.
- Every task has a **deadline** and review on completion.
- Encourage problem-solving before escalating blockers.
- Trackers must be updated regularly.
- Work progress should be committed and shared on GitHub.

Checklist for Task Completion

- Code should run **without errors**.
- Work must meet all **acceptance criteria**.
- Each task is **reviewed and tested** by everyone
- Documentation should be updated with the task.
- Final review by all team members before submission.

THE STARTUP FOUNDER PERSONA

Name: Alice Johnson

Age: 28

Gender: Female

Occupation: Startup Founder

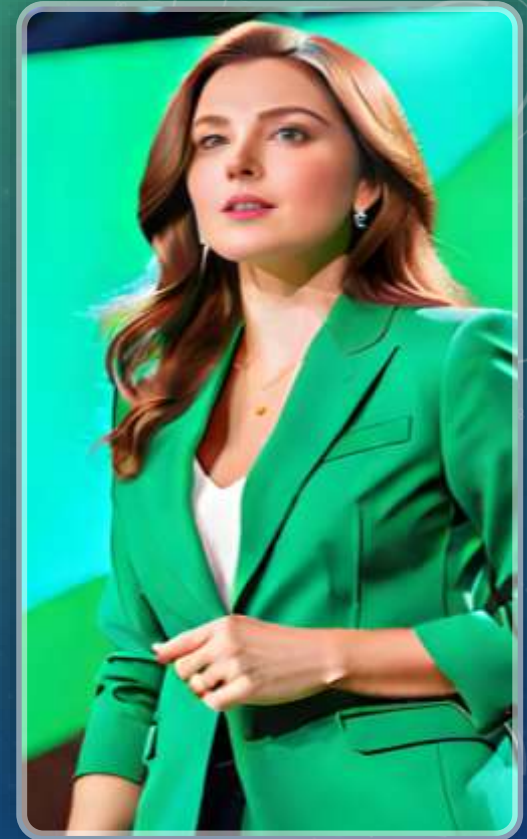
Alice is an ambitious entrepreneur developing a sustainable fashion brand. She wants to raise money to bring her eco-friendly products to market but struggles with high fees and limited visibility on traditional crowdfunding platforms.

Challenges:

- Struggles to gain trust from contributors due to lack of transparency.
- Centralized platforms charge high fees, cutting into her funding.
- Limited control over how her campaign is presented.

Goals:

- Raise funds fairly and transparently.
- Build credibility with backers.
- Ensure contributors see exactly how their money is being used.



THE INVESTOR PERSONA

Name: Bob Smith

Age: 35

Gender: Male

Occupation: Investor

Bob is a passionate investor who likes to fund innovative projects. He wants to support creators but often worries about whether his money is being used properly and feels disconnected after contributing.

Challenges:

- Unsure if funds are spent transparently
- No refunds if a project fails
- Has no control once money is invested

Goals:

- Invest with full transparency and security
- Receive refunds if funding goals aren't met
- Have a say in how project funds are used through voting



THE COMMUNITY MEMBER PERSONA

Name: Cathy Lee

Age: 22

Gender: Female

Occupation: College Student / Community Member

Cathy is an active member of online communities and DAOs. She enjoys contributing to causes but wants her voice to count, not just her money.

Challenges:

- No say in how project funds are allocated
- Hesitant to contribute without accountability
- Finds most platforms profit-driven and restrictive

Goals:

- Participate in voting and decision-making
- Support transparent, community-driven projects
- Learn about blockchain while backing meaningful causes



MVP

- **Campaign Creation:** Allow users to launch funding campaigns with metadata (title, goal, deadline).
- **Contribution Logic:** Accept ETH contributions and track them per user.
- **Refund Mechanism:** Automatically refund contributors if the goal isn't met.
- **Governance Voting:** Let contributors vote on fund allocation or milestones.



TOOLS & TECHNOLOGIES

- **Blockchain:** Ethereum + Solidity
- **Development:** Remix IDE, Hardhat
- **Network:** MetaMask, Sepolia Test Network, Sepolia ETH Faucets
- **Deployment & Testing:** Hardhat
- **Collaboration Tools:** GitHub, VS Code



ALGORITHMS & LOGIC

Contribution Algorithm

Algorithm Steps

1. Ensure the current time is before the deadline.
2. Check that the contribution meets the minimum amount.
3. Register new contributors.
4. Update contributor balance and total raised amount.

```
function sendEth() public payable {  
    require(block.timestamp < deadline);  
    require(msg.value >= minContribution);  
  
    if (contributors[msg.sender] == 0) {  
        totcontributors++;  
    }  
    contributors[msg.sender] += msg.value;  
    raisedAmount += msg.value;  
}
```



ALGORITHMS & LOGIC

Refund Algorithm

Algorithm Steps

1. Check that the deadline has passed.
2. Ensure the target was not reached.
3. Verify caller is a contributor.
4. Return contributed Ether.

```
function refund() public {  
    require(block.timestamp > deadline);  
    require(raisedAmount < target);  
    require(contributors[msg.sender] > 0);  
  
    uint amount = contributors[msg.sender];  
    contributors[msg.sender] = 0;  
    payable(msg.sender).transfer(amount);  
}
```



ALGORITHMS & LOGIC

Request Approval & Payment Algorithm

Algorithm Steps

1. Manager creates a spending request.
2. Contributors vote (one vote per contributor).
3. After deadline, check majority approval.
4. Transfer funds and mark request completed.



```
function makePay(uint256 _requestNo) public {
    Request storage r = request[_requestNo];

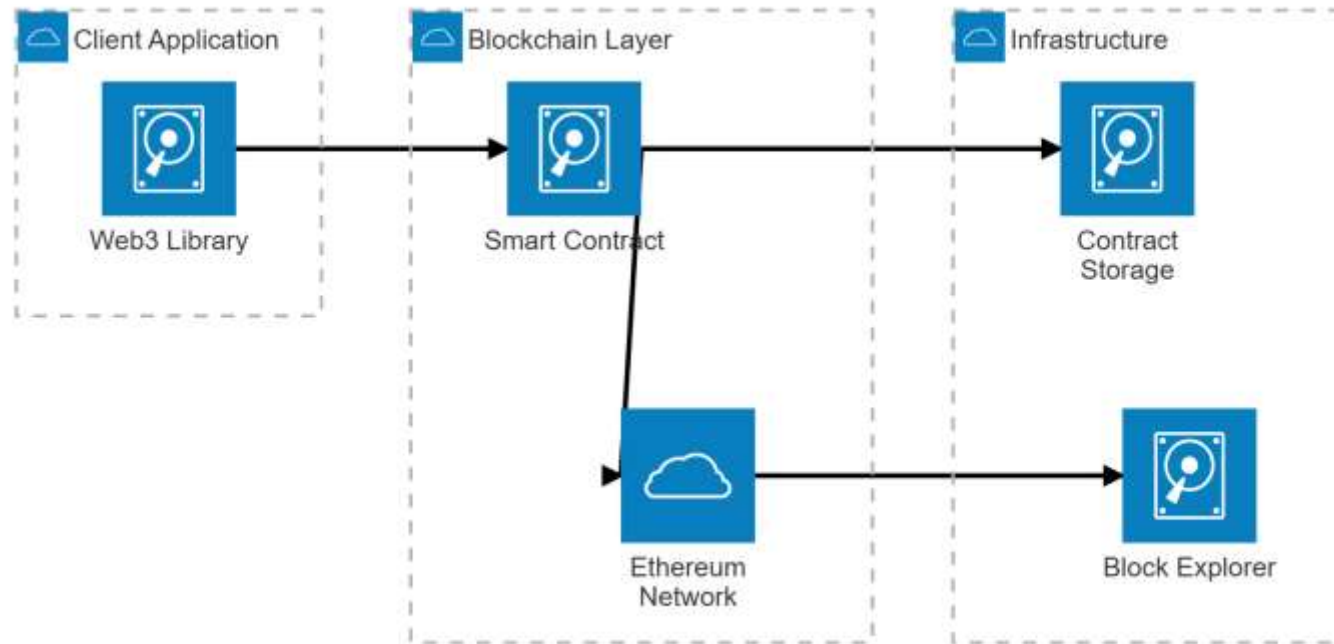
    require(block.timestamp > deadline);
    require(raisedAmount >= target);
    require(!r.completed);
    require(r.totVoters > totcontributors / 2);

    r.recipient.transfer(r.value);
    r.completed = true;
}
```

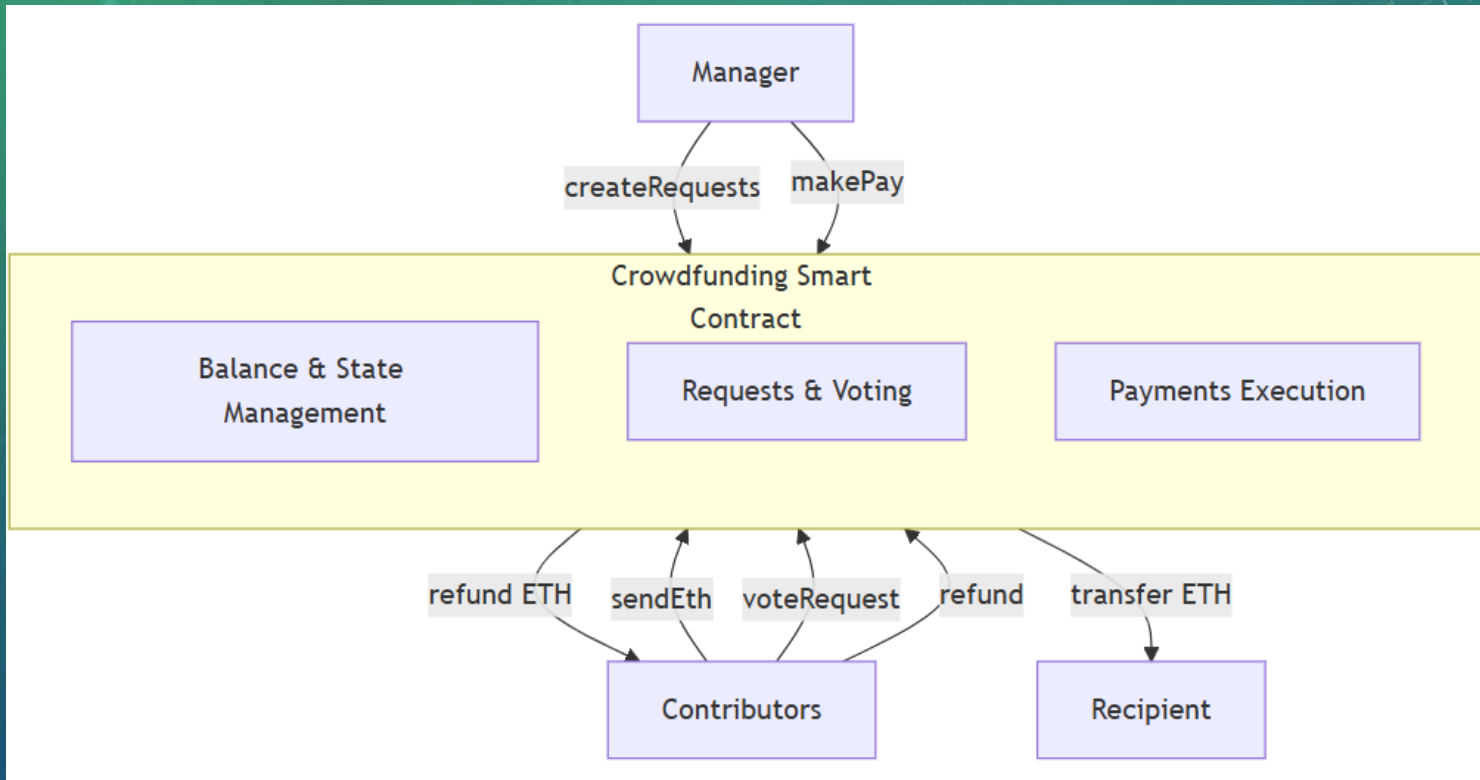
Diagrams

The background is a gradient of green and blue, transitioning from a lighter green at the top to a darker blue at the bottom. It is covered with a fine pattern of white dots. Several faint, white circular diagrams are scattered across the background. These diagrams include concentric circles, dashed lines, and arrows indicating a clockwise or counter-clockwise direction. One prominent diagram in the upper right corner features a circular scale with numerical markings from 0 to 210 in increments of 10, and an arrow pointing counter-clockwise. Another diagram in the lower right shows concentric circles with a dashed outer ring and an arrow pointing clockwise. A third diagram in the lower left shows a partial circle with an arrow pointing clockwise.

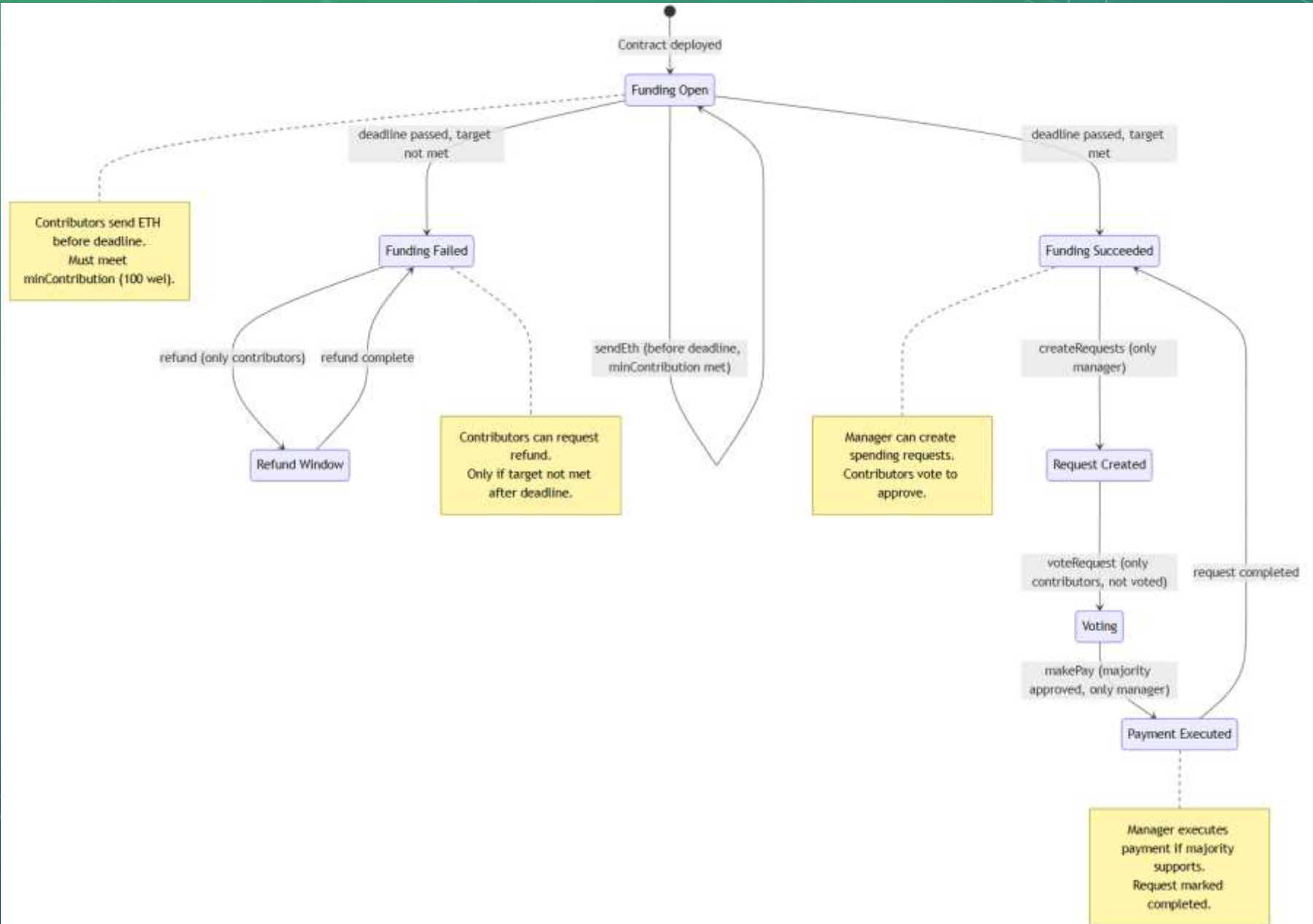
ARCHITECTURE DIAGRAM



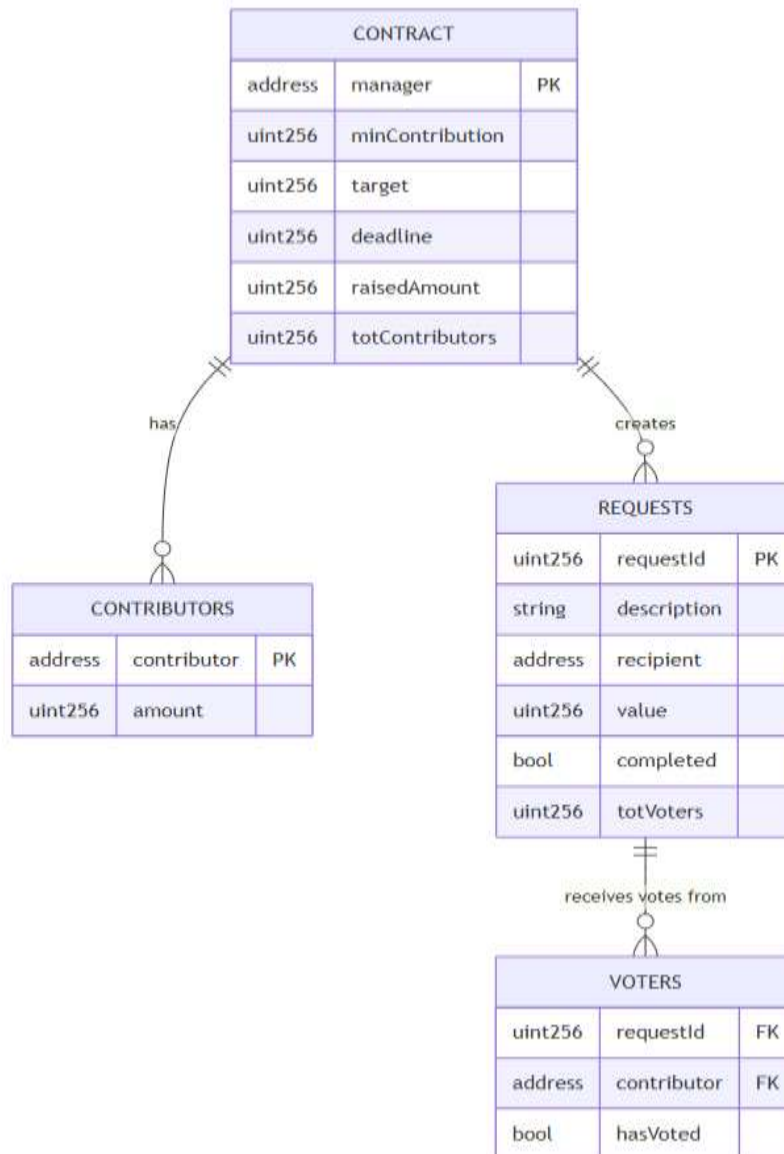
CONTEXT DIAGRAM



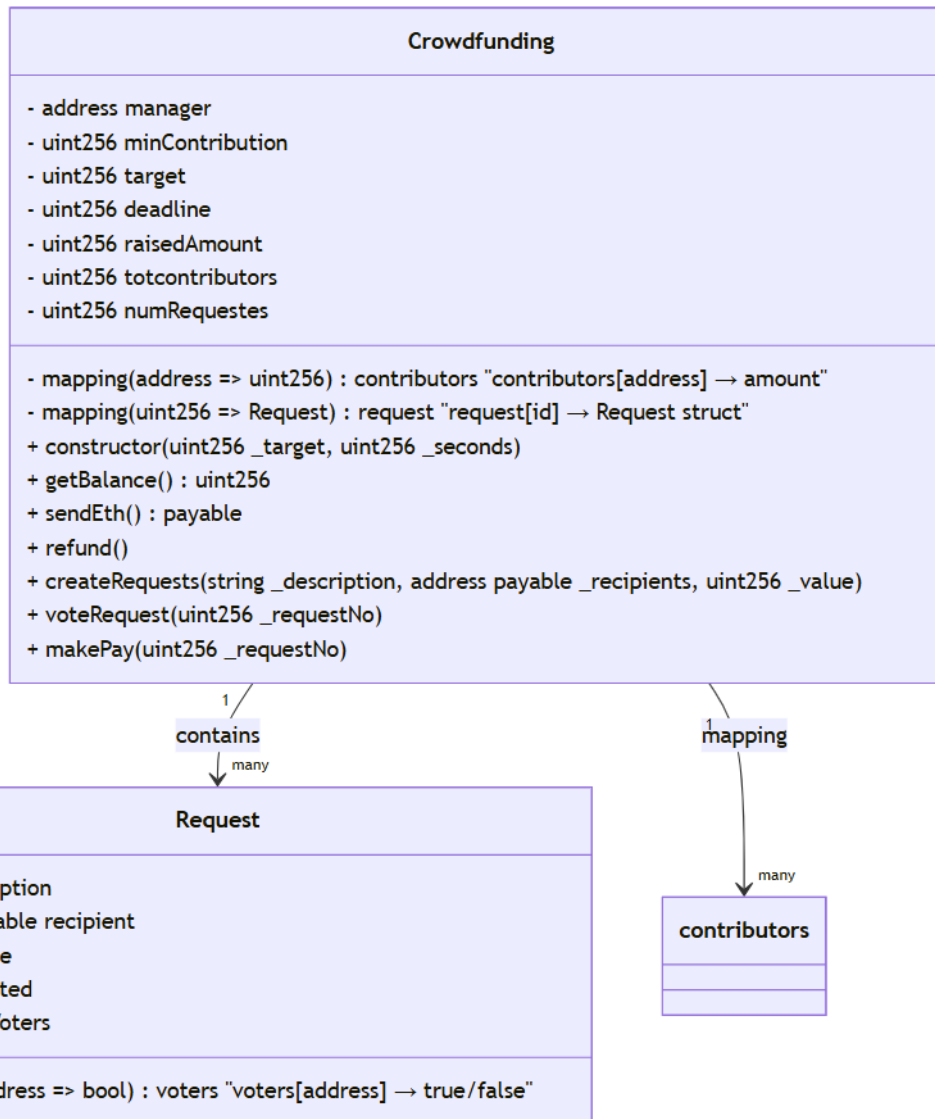
STATE DIAGRAM



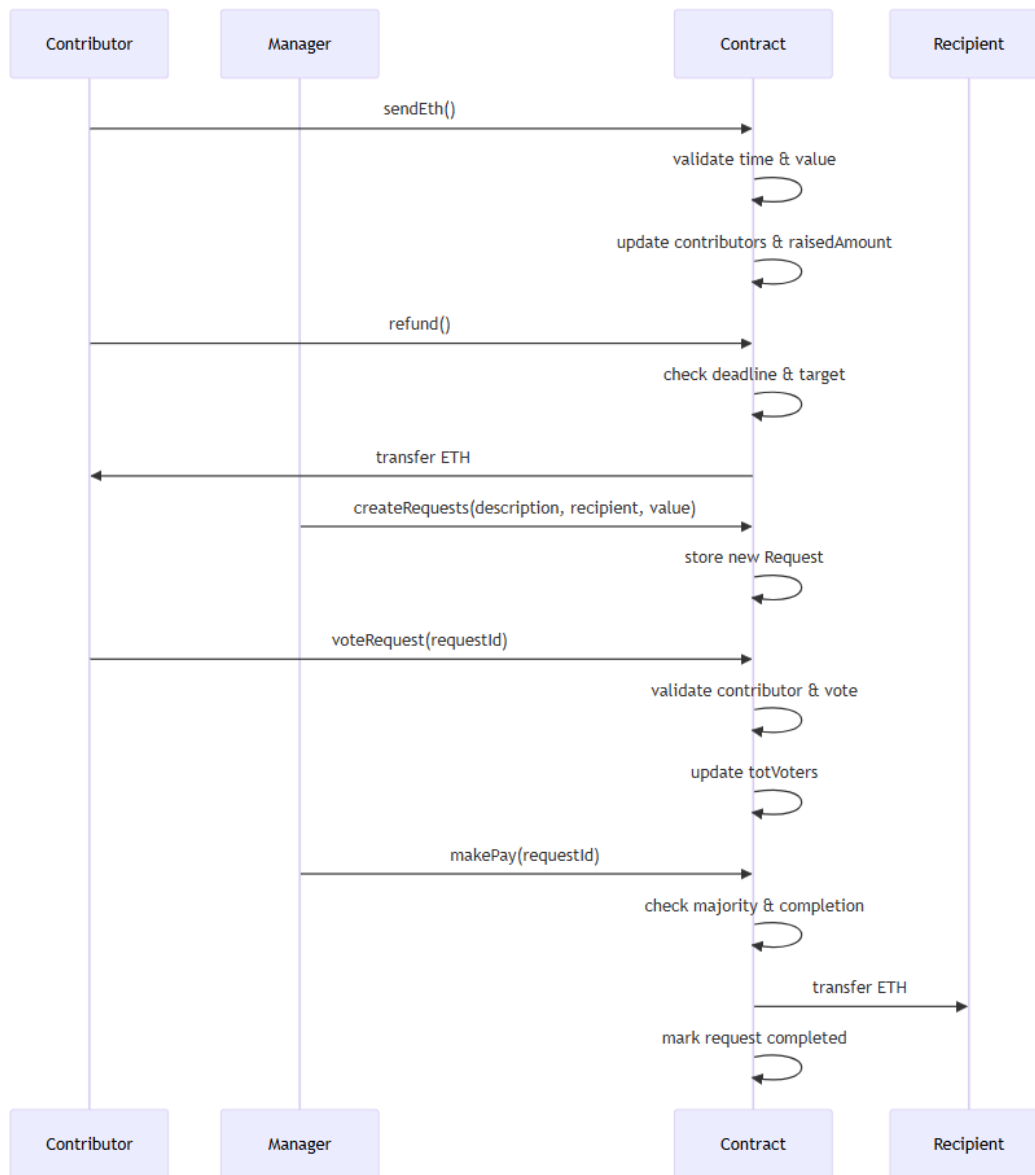
ER DIAGRAM



CLASS DIAGRAM



SEQUENCE DIAGRAM



Sprint 2 Recap (21 pts)

Goal: Implement fund management features in the crowdfunding system.

User Stories Completed:

- **US_04 – Create Spending Requests (Manager)**
Manager can propose how raised funds are used by creating requests with description, recipient, and amount.
- **US_05 – Vote on Spending Requests (Contributor)**
Contributors can vote on requests (one vote per contributor) to influence fund usage.
- **US_06 – Release Funds After Approval (Manager)**
Funds can be released only if a request has majority approval, ensuring responsible fulfillment.

Outcome:

Voting-based governance implemented; contributors control spending; managers can propose requests securely.

PRODUCT BACKLOG



No.	User Story	Acceptance Criteria	Feature Name	Component	Story Points
US_01	As a campaign manager, I want to create a new campaign with a target and deadline so that I can raise funds.	Given I am the manager, When I deploy the contract with a target and deadline, Then the campaign is initialized and I am set as owner.	Campaign Creation	Smart Contract	8
US_02	As a contributor, I want to contribute funds to a campaign so that I can support causes I believe in.	Given the campaign is active, When I send ETH above the minimum, Then my contribution is recorded and the total raised updates.	Campaign Contribution	Smart Contract	5
US_03	As a contributor, I want to get a refund if the campaign fails so that I don't lose my funds.	Given the deadline has passed and the target was not met, When I request a refund, Then I receive my ETH and my record is cleared.	Refund Processing	Smart Contract	5
US_04	As a manager, I want to create spending requests so that I can propose how to use the raised funds.	Given I am the manager, When I create a request with description, amount, and recipient, Then the request is stored with a unique ID.	Request Creation	Governance	8
US_05	As a contributor, I want to vote on spending requests so that I can influence how funds are used.	Given I am a contributor, When I vote on a request I haven't voted on before, Then my vote is recorded and vote count increases.	Request Voting	Governance	5

No.	User Story	Acceptance Criteria	Feature Name	Component	Story Points
US_06	As a manager, I want to release funds after majority approval so that requests can be fulfilled responsibly.	Given the deadline has passed and the target is met, When a request has majority support and I execute it, Then ETH is transferred and request marked completed.	Fund Release	Governance	8
US_07	As a user, I want to view all campaigns with details so that I can choose which to support.	Given campaigns exist, When I open the campaign list, Then I see each campaign's goal, raised amount, and current status.	Campaign Listing	Data Display	5
US_08	As a user, I want to see campaign status so that I know whether it's active, successful, or failed.	Given a campaign has a deadline and target, When time passes or funds are raised, Then the status updates to Active, Successful, or Failed.	Campaign Status	Data Display	3
US_09	As a system, I want to validate minimum contributions so that campaigns remain fair and secure.	Given a contribution is below the minimum, When a user tries to send ETH, Then the transaction is rejected and an error message is shown.	Validation Logic	Contract Validation	2

PRODUCT BACKLOG OVERVIEW

Sprint 1 (15 pts)	Sprint 2 (21 pts)	Sprint 3 (13 pts)
US_01: As a campaign manager, I want to create a new campaign with a target and deadline so that I can raise funds.	US_04: As a manager, I want to create spending requests so that I can propose how to use the raised funds.	US_03: As a contributor, I want to get a refund if the campaign fails so that I don't lose my funds
US_02: As a contributor, I want to contribute funds to a campaign so that I can support causes I believe in	US_05: As a contributor, I want to vote on spending requests so that I can influence how funds are used.	US_07: As a user, I want to view all campaigns with details so that I can choose which to support.
US_09: As a system, I want to validate minimum contributions so that campaigns remain fair and secure.	US_06: As a manager, I want to release funds after majority approval so that requests can be fulfilled responsibly.	US_08: As a user, I want to see campaign status so that I know whether it's active, successful, or failed.

SPRINT SUMMARY

Sprint 1 Summary (Oct 14 – Oct 21)

Sprint Goal:

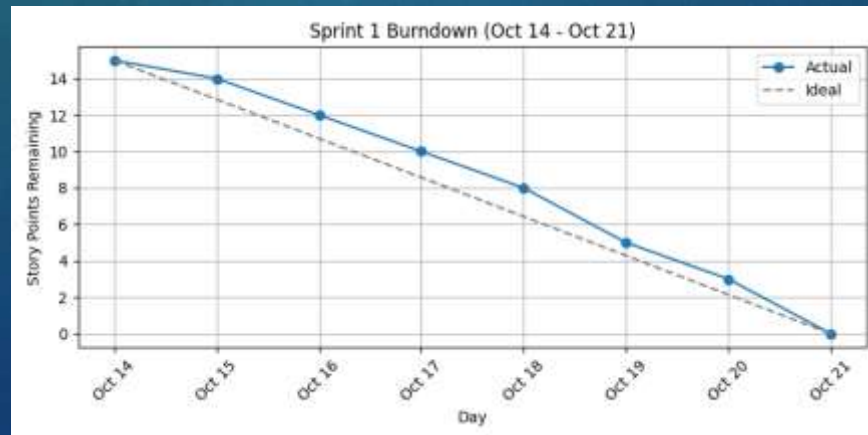
Enable basic campaign creation and contribution functionality while enforcing fair contribution rules.

Completed User Stories (15 pts):

- **US_01:** Campaign creation with target and deadline (Completed)
- **US_02:** Contribution to campaigns (Completed)
- **US_09:** Validation of minimum contributions (Completed)

Sprint Outcome:

All planned user stories were completed. The campaign manager can now create campaigns with specific targets and deadlines. Contributors can fund campaigns while the system ensures minimum contribution rules are enforced.



SPRINT SUMMARY

Sprint 2 Summary (Oct 22 – Nov 12)

Sprint Goal:

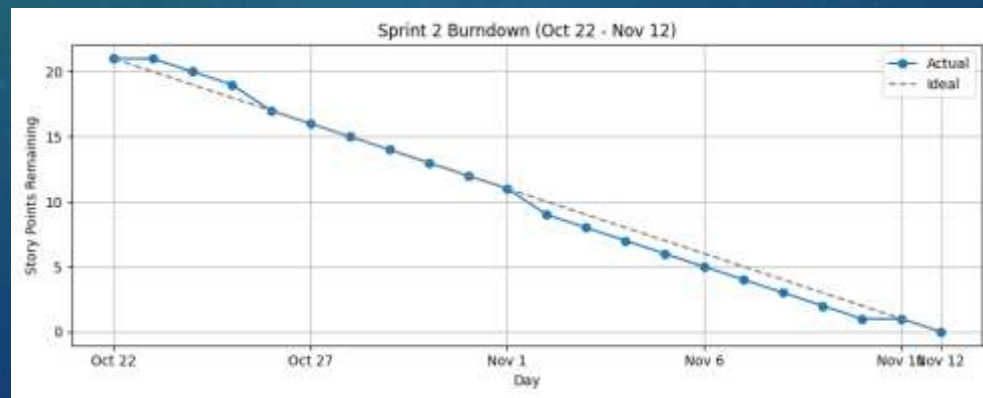
Implement spending management, including requests, contributor voting, and fund release after approval.

Completed User Stories (21 pts):

- **US_04:** Manager can create spending requests (Completed)
- **US_05:** Contributors can vote on spending requests (Completed)
- **US_06:** Manager can release funds after majority approval (Completed)

Sprint Outcome:

Managers can now propose spending requests and contributors can participate in decision-making. Funds are only released after majority approval, ensuring responsible handling of campaign resources.



SPRINT 3

SPRINT 3 BACKLOG

No.	User Story	Acceptance Criteria	Feature Name	Component	Story Points	Assigned to
US_03	As a contributor, I want to get a refund if the campaign fails so that I don't lose my funds	Given the deadline has passed and the target was not met, When I request a refund, Then I receive my ETH and my record is cleared.	Refund Processing	Smart Contract	5	Ashwanth Yashwanth
US_07	As a user, I want to view all campaigns with details so that I can choose which to support.	Given campaigns exist, When I open the campaign list, Then I see each campaign's goal, raised amount, and current status.	Campaign Listing	Data Display	5	Nyi Nyi Raj
US_08	As a user, I want to see campaign status so that I know whether it's active, successful, or failed.	Given a campaign has a deadline and target, When time passes or funds are raised, Then the status updates to Active, Successful, or Failed.	Campaign Status	Data Display	3	Ashwitha Naveen

SPRINT 3 BACKLOG TESTCASES

Feature to Test	User Story	Test Description	Test Data	Expected Result	Actual Result	Pass/Fail	Assigned To
Refund Processing	US_03	Request refund when campaign failed	Contributor: 0xABC123, Raised: 5 ETH, Target: 10 ETH, Deadline passed	ETH returned; record cleared	ETH successfully returned; record cleared	Pass	Ashwanth / Yashwanth
Refund Processing	US_03	Request refund when campaign active	Contributor: 0xABC123, Raised: 5 ETH, Target: 10 ETH, Deadline not passed	Error: "You are not able take refund"	Error correctly shown	Pass	Ashwanth / Yashwanth
Refund Processing	US_03	Request refund without contribution	Contributor: 0xDEF456, No contributions	Error: "You are not contributor.."	Error correctly shown	Pass	Ashwanth / Yashwanth
Campaign Listing	US_07	View all campaigns	N/A – at least one campaign exists	List shows campaigns with goal, raised amount, status	Campaign list displays correctly	Pass	Nyi Nyi / Raj
Campaign Listing	US_07	Campaign list empty	N/A – no campaigns exist	Message: "No campaigns available"	Page displays message correctly	Pass	Nyi Nyi / Raj
Campaign Status	US_08	Status updates to Active	Campaign: "Plant Trees", Raised: 2 ETH, Target: 10 ETH, Deadline not reached	Status shows "Active"	Status correctly shows "Active"	Pass	Ashwitha / Naveen
Campaign Status	US_08	Status updates to Successful	Campaign: "Plant Trees", Raised: 10 ETH, Target: 10 ETH, Deadline not reached	Status shows "Successful"	Status correctly shows "Successful"	Pass	Ashwitha / Naveen
Campaign Status	US_08	Status updates to Failed	Campaign: "Plant Trees", Raised: 5 ETH, Target: 10 ETH, Deadline passed	Status shows "Failed"	Status correctly shows "Failed"	Pass	Ashwitha / Naveen

STORIES COMPLETED

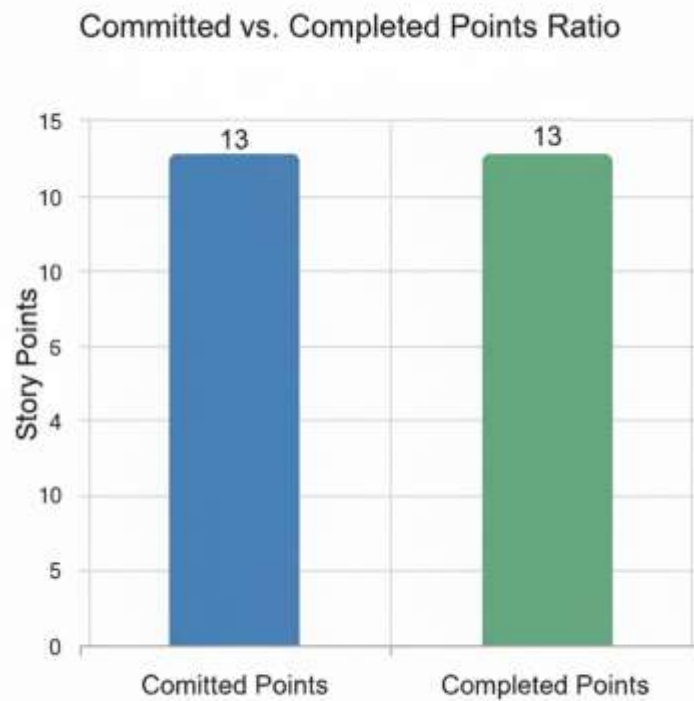
Backlog No	User Story	Completed Date (Start → Finish)	Sub-Tasks	Story Points	Completed Points
US_03	As a contributor, I want to get a refund if the campaign fails so that I don't lose my funds	Nov 15 → Nov 20	• Implement refund logic in smart contract • Validate deadline & target conditions • Clear contributor record after refund • Test refund success & failure cases	5	5
US_07	As a user, I want to view all campaigns with details so that I can choose which to support	Nov 21 → Nov 24	• Fetch campaign data from contract • Display goal & raised amount • Display campaign status • Handle empty campaign list	5	5
US_08	As a user, I want to see campaign status so that I know whether it's active, successful, or failed	Dec 2 → Dec 4	• Define status rules (Active/Successful/Failed) • Compare deadline and target values • Update status dynamically • Verify status with test data	3	3

METRICS: SPRINT 2

TEAM VELOCITY

User Story ID	User Story Description	Story Points	Status	Completed Points
US_03	As a contributor, I want to get a refund if the campaign fails so that I don't lose my funds	5	Completed	5
US_07	As a user, I want to view all campaigns with details so that I can choose which to support	5	Completed	5
US_08	As a user, I want to see campaign status so that I know whether it's active, successful, or failed	3	Completed	3

COMMITTED/COMPTETED RATIO



METRICS: BURNDOWN CHART



SPRINT 3 RETROSPECTIVE

What Went Well

- All planned user stories (US_03, US_07, US_09) were completed successfully.
- 100% of test cases passed, including edge cases for refunds, deadlines, and contributions.
- Contributions, deadlines, and campaign totals tracked accurately.
- Smooth team collaboration and coordination for tasks, merging, and testing.
- Significant learning in smart contract logic, Solidity features, and testing practices.

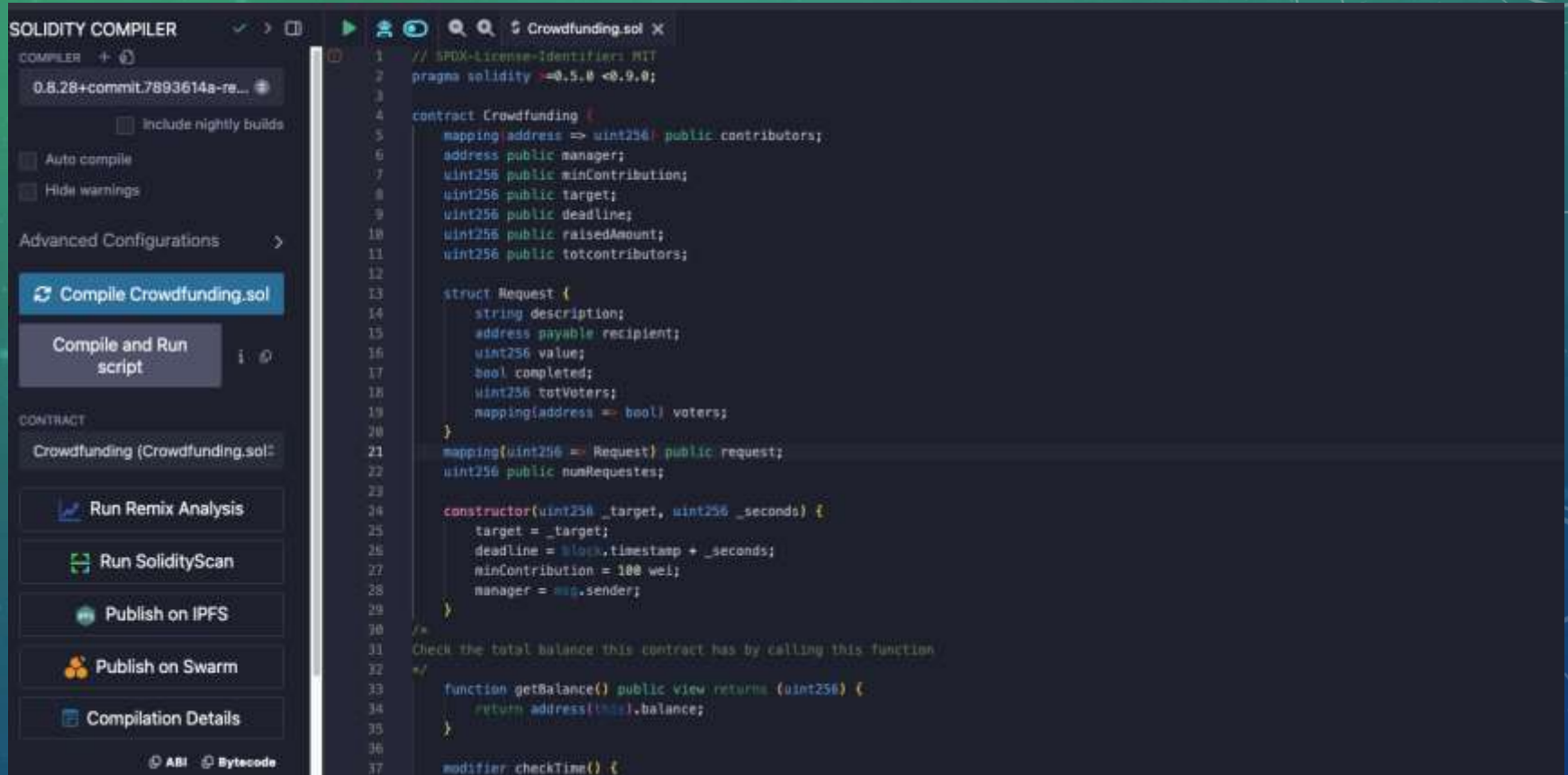
What Could Be Improved

- Some tasks, especially complex test scenarios, took longer than estimated.
- Documentation for smart contract deployment and usage could be more detailed.
- Rare edge cases were discovered later than ideal; earlier identification would save time.

Action Items

- Plan comprehensive tests early to cover all scenarios.
- Improve documentation for deployment, usage, and maintenance.
- Refine task estimation using historical sprint data.
- Conduct peer reviews to identify uncommon edge cases early.
- Prepare smart contracts for future front-end integration.

PROJECT DEMO



The image shows a screenshot of the Solidity Compiler interface. On the left, there is a sidebar with the following sections:

- SOLIDITY COMPILER**: Includes a dropdown for the compiler version (0.8.28+commit.7893614a-re...), checkboxes for "Include nightly builds", "Auto compile", and "Hide warnings", and a link to "Advanced Configurations".
- CONTRACT**: Shows the selected contract "Crowdfunding (Crowdfunding.sol)".
- Actions**: Buttons for "Run Remix Analysis", "Run SolidityScan", "Publish on IPFS", "Publish on Swarm", and "Compilation Details".
- ABI / Bytecode**: Tabs at the bottom of the sidebar.

The main area displays the Solidity code for the "Crowdfunding.sol" contract. The code is as follows:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.5.0 <0.9.0;
3
4 contract Crowdfunding {
5     mapping(address => uint256) public contributors;
6     address public manager;
7     uint256 public minContribution;
8     uint256 public target;
9     uint256 public deadline;
10    uint256 public raisedAmount;
11    uint256 public totContributors;
12
13    struct Request {
14        string description;
15        address payable recipient;
16        uint256 value;
17        bool completed;
18        uint256 totVoters;
19        mapping(address => bool) voters;
20    }
21    mapping(uint256 => Request) public request;
22    uint256 public numRequests;
23
24    constructor(uint256 _target, uint256 _seconds) {
25        target = _target;
26        deadline = block.timestamp + _seconds;
27        minContribution = 100 wei;
28        manager = msg.sender;
29    }
30    /*
31    Check the total balance this contract has by calling this function
32    */
33    function getBalance() public view returns (uint256) {
34        return address(this).balance;
35    }
36
37    modifier checkTime() {
```

WIKI PAGE LINK

<https://github.com/htmwf2025-Team2/wiki>



Thank You!